

COURSE MATERIAL



# Module 1: Introduction to Databases & RDBMS Fundamentals

Module 1 - 6 Hours

Introduction to Databases & RDBMS Fundamentals

# Module 1 at a glance

|                               |                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------|
| <b>Duration</b>               | 6 hours (3 hours theory, 3 hours lab)                                                  |
| <b>Prerequisites</b>          | Basic programming, basic command line                                                  |
| <b>You will build</b>         | A running PostgreSQL, Oracle XE and MySQL install, loaded with the ICICI Bank database |
| <b>You will carry forward</b> | The <code>icicibank</code> database. Modules 2 to 12 all use it.                       |

## Learning outcomes

By the end of this module you can:

1. Explain what a database management system does that a file system cannot.
2. State the difference between a DBMS and an RDBMS, using ICICI Bank examples.
3. Describe Oracle, PostgreSQL and MySQL, and justify which fits a given banking workload.
4. Distinguish OLTP from OLAP and explain why banks keep them on separate servers.
5. Draw and explain the architecture of a database server.
6. Define each of the four ACID properties and demonstrate them in SQL.

## The business problem

### **Ticket ICICI-4417 - Daily reconciliation takes 3 days**

The Fort Mumbai branch keeps account balances in a spreadsheet. Andheri East keeps its own copy. When a customer moves money between branches the numbers stop matching, and a clerk fixes them by hand. Explain what a proper database gives us, and prove it.

Everything in this module answers some part of that ticket. Keep it in mind as you read.

# 1. From files to databases

## 1.1 How banks stored data before databases

Before the mid-1990s a branch of ICICI Bank was, in data terms, an island. The branch kept a ledger, then a spreadsheet, and it was the only authority on its own customers' balances. Head office received a summary at month end.

This is the **file-based approach**: each application owns its own files, in its own format, and there is no shared engine in between.

```
Fort Mumbai branch          Andheri East branch
-----
FORT_CUSTOMERS.xls           ANDHERI_CUSTOMERS.xls
FORT_BALANCES.xls           ANDHERI_BALANCES.xls
FORT_LOANS.xls               ANDHERI_LOANS.xls

Customer 1001 appears in BOTH files, with two different addresses,
and nothing in the world says which one is correct.
```

## 1.2 The six problems

These are not academic. Each one has caused a real banking incident somewhere.

| Problem                         | Meaning                                     | At ICICI Bank                                               |
|---------------------------------|---------------------------------------------|-------------------------------------------------------------|
| <b>Data redundancy</b>          | The same fact stored in more than one place | Customer 1001's address sits in four branch files           |
| <b>Data inconsistency</b>       | Redundant copies disagree                   | Fort says the balance is 52,000, Andheri says 47,500        |
| <b>No concurrent access</b>     | Two people cannot safely edit at once       | Two clerks open the workbook, the second save silently wins |
| <b>No integrity enforcement</b> | Nothing rejects impossible data             | A clerk types a balance of -900,000 and the file accepts it |
| <b>Poor security</b>            | Access is all-or-nothing at the file level  | Whoever opens the folder reads every salary account         |
| <b>No recovery</b>              | A failure loses work with no way back       | Power cut during a save corrupts the workbook               |

## 1.3 What a DBMS is

A **Database Management System** is software that stores data and mediates every single access to it. Applications never open the data files themselves. They ask the engine, and the engine decides.

That indirection is what buys the six guarantees:

- **Storage and retrieval** - the application says what it wants, not how to find it.
- **Concurrency control** - thousands of ATMs, UPI apps and tellers work simultaneously without corrupting each other's work.
- **Integrity enforcement** - rules like "a balance may not be negative" live in the database, so no application can bypass them.

- **Security and authorisation** - a teller and an auditor connect to the same database and see different things.
- **Backup and recovery** - a crash is survivable, and the recovery point is measured in seconds.
- **A query language** - questions are expressed in SQL rather than written as file-scanning code.

**The key idea:** a DBMS moves responsibility from every application into one engine. Ten applications no longer need ten correct implementations of "do not allow a negative balance". They need one, in the database.

## 1.4 DBMS versus RDBMS

A **Relational** DBMS is a DBMS that organises data as tables and lets you declare the relationships between them.

| Aspect            | DBMS                                               | RDBMS                                 |
|-------------------|----------------------------------------------------|---------------------------------------|
| Data organised as | Files, hierarchies, or a network of linked records | Tables of rows and columns            |
| Relationships     | Written into application code by the programmer    | Declared as primary and foreign keys  |
| Rules enforced by | The application                                    | The database engine                   |
| Normalization     | Not a concept                                      | Central to the design method          |
| Transactions      | Rare or limited                                    | Full ACID support                     |
| Query language    | Proprietary or none                                | SQL                                   |
| Examples          | dBase, IMS, XML stores, file managers              | Oracle, PostgreSQL, MySQL, SQL Server |

Every RDBMS is a DBMS. Not every DBMS is relational. Redis, which you meet in Module 7, and MongoDB are both DBMSs and neither is relational.

## 2. The relational model

### 2.1 The vocabulary

Relational theory has formal terms and everyday terms. You will hear both.

| Formal term | Everyday term               | ICICI Bank example                                                |
|-------------|-----------------------------|-------------------------------------------------------------------|
| Relation    | Table                       | <code>account</code>                                              |
| Tuple       | Row                         | One real savings account                                          |
| Attribute   | Column                      | <code>balance</code>                                              |
| Degree      | Number of columns           | <code>account</code> has 10                                       |
| Cardinality | Number of rows              | 36 in the training dataset                                        |
| Domain      | Allowed values for a column | <code>account_type</code> is one of SAVINGS, CURRENT, SALARY, NRI |

### 2.2 A table you will use for 120 hours

```

account
+-----+-----+-----+-----+-----+-----+
| account_id | account_number | customer_id | account_type | balance | status |
+-----+-----+-----+-----+-----+-----+
| 9001      | ICIC0000009001 | 1001       | CURRENT     | 35859.46 | DORMANT |
| 9002      | ICIC0000009002 | 1002       | SALARY      | 43522.54 | ACTIVE  |
| 9003      | ICIC0000009003 | 1003       | SAVINGS     | 341244.80 | ACTIVE  |
+-----+-----+-----+-----+-----+-----+
      ^           ^           ^
      |           |           |
primary key  candidate key  foreign key -> customer.customer_id
  
```

### 2.3 Keys

You will study keys properly in Module 4. For now, recognise four of them.

- **Primary key** - the column, or set of columns, that uniquely identifies a row. It can never be NULL and never repeats. `account.account_id`.
- **Candidate key** - any column that could legitimately have been chosen as the primary key. `account.account_number` is unique, so it qualifies.
- **Foreign key** - a column whose value must exist as a primary key in another table. `account.customer_id` must match a real `customer`. This is what stops an account from belonging to a customer who does not exist.
- **Composite key** - a primary key made of two or more columns together. A table of daily branch balances might be keyed on (`branch_id`, `balance_date`).

### 2.4 Codd's rules, point by point

E. F. Codd defined the relational model at IBM in 1970. In 1985 he published thirteen rules, numbered 0 to 12, that a product must satisfy before it may honestly call itself relational. He wrote them because vendors were relabelling older products as "relational" without changing them.

**No commercial database satisfies all thirteen.** That is not a failure of the rules. They are a yardstick, and knowing where a product falls short tells you exactly where you will have to be careful. Each rule below is given with what it demands, what it means at ICICI Bank, and how real databases actually behave.

### The rules at a glance

| #  | Name                      | In one line                                               |
|----|---------------------------|-----------------------------------------------------------|
| 0  | Foundation                | Everything must be done relationally. No side doors.      |
| 1  | Information               | All data is values in tables. Nothing hidden elsewhere.   |
| 2  | Guaranteed access         | Table plus key plus column always reaches a value.        |
| 3  | Systematic NULL           | One NULL, for every type, meaning "no value here".        |
| 4  | Active online catalog     | The catalog is tables, queried with ordinary SQL.         |
| 5  | Comprehensive sublanguage | One language does definition, data, rules and control.    |
| 6  | View updating             | Every theoretically updatable view is actually updatable. |
| 7  | Set level operations      | Insert, update and delete work on sets, not one row.      |
| 8  | Physical independence     | Change the storage, change no application.                |
| 9  | Logical independence      | Change the design, change no application.                 |
| 10 | Integrity independence    | Rules live in the database, not in application code.      |
| 11 | Distribution independence | Move data between servers, change no application.         |
| 12 | Non-subversion            | No low-level route may bypass the rules.                  |

## Rule 0 - The foundation rule

**Demands:** a system that claims to be relational must manage its data entirely through its relational capabilities.

**At ICICI Bank:** there is no supported way to change a balance except through SQL. You cannot open the data file in an editor, and no utility lets you poke a byte. Everything goes through the engine, which is what the client-server picture in section 5.1 shows.

**Reality:** honoured by every serious product. It is the rule that makes all the others enforceable.

## Rule 1 - The information rule

**Demands:** all information, including metadata, is represented in exactly one way, as values in columns within rows of tables.

**At ICICI Bank:** a balance is a value in `account.balance`. A column's data type is also a value, in a catalog table. There is no second, hidden mechanism where some facts live.

**Reality:** honoured, with one asterisk. Comments, file locations and some configuration sit outside tables in every product.

## Rule 2 - The guaranteed access rule

**Demands:** every single value is reachable by naming its table, its primary key value and its column. No pointers, no positions, no navigation path.

**At ICICI Bank:** to reach Aarav Sharma's balance you need `account`, key `9001`, column `balance`. Nothing else. You never need to know which file, which block, or which row came first.

```
SELECT balance FROM account WHERE account_id = 9001;
```

**Reality:** honoured. This rule is why a table without a primary key is a defect rather than a style choice, and it is the reason Module 4 insists every table has one.

## Rule 3 - Systematic treatment of NULL

**Demands:** NULL must be supported for representing missing or inapplicable information, in a way that does not depend on the data type and is distinct from zero, from an empty string, and from any real value.

**At ICICI Bank:** `fixed_deposit.nominee_name` is NULL when no nominee has been recorded. That is not the same as an empty name. `account_txn.narration` may be NULL. A NULL interest rate is not a zero interest rate, and a bank that confuses the two pays or charges the wrong money.

**Reality:** honoured, and it is the single largest source of quiet bugs on this programme. NULL comparisons return unknown rather than true or false, which is why `IS NULL` exists, why `NOT IN` breaks against a NULL, and why `COUNT(column)` and `COUNT(*)` disagree. Module 2 and Module 3 both return to this.

## Rule 4 - Active online catalog based on the relational model

**Demands:** the description of the database is itself stored as tables, and is queryable with the same language used for the data.

**At ICICI Bank:** you find out what columns `loan` has by querying `information_schema.columns`, using ordinary `SELECT`. No separate tool, no separate language.

```
SELECT column_name, data_type, is_nullable
FROM   information_schema.columns
WHERE  table_name = 'loan';
```

**Reality:** honoured everywhere, and directly useful. Module 9 builds the audit reports out of catalog queries, and Module 11 builds the entire migration assessment that way.

## Rule 5 - The comprehensive data sublanguage rule

**Demands:** there must be at least one language that can express data definition, data manipulation, integrity constraints, authorisation, views and transaction boundaries, and it must work both interactively and from inside a program.

**At ICICI Bank:** SQL does all of it. `CREATE TABLE`, `SELECT`, `CHECK`, `GRANT`, `CREATE VIEW`, `COMMIT`. The same text runs in `psql` and inside the Python code in Module 7.

**Reality:** honoured, although each vendor has extended SQL in incompatible directions. That divergence is precisely what makes Module 11's Oracle to PostgreSQL conversion a project rather than a copy.

## Rule 6 - The view updating rule

**Demands:** every view that is theoretically updatable must be updatable by the system.

**At ICICI Bank:** a view showing active accounts for one branch should accept an update to a balance and pass it through to `account`.

**Reality: commonly broken.** A view over a single table with no aggregation is usually updatable. Add a join, a `GROUP BY`, a `DISTINCT` or a window function and the engine refuses, even where the update is theoretically unambiguous. PostgreSQL offers `INSTEAD OF` triggers as a manual way round it. This is the rule real products break most often.

## Rule 7 - High level insert, update and delete

**Demands:** the system must treat a set of rows as a single operand, for retrieval and equally for insert, update and delete.

**At ICICI Bank:** the month-end job that matures every eligible deposit is one statement, not a loop.

```
UPDATE fixed_deposit
SET    status = 'MATURED'
WHERE  status = 'ACTIVE' AND maturity_date <= DATE '2026-06-30';
```

**Reality:** honoured, and routinely thrown away by developers who write a cursor loop instead. Set-at-a-time against row-at-a-time is a recurring performance theme in Module 5 and again in Module 10's trigger section.

## Rule 8 - Physical data independence

**Demands:** applications must not be affected by changes in how data is physically stored or accessed.

**At ICICI Bank:** moving `account_txn` to faster storage, adding an index, or partitioning it by month changes no application code and no SQL.

**Reality:** honoured for storage and indexing. Not honoured for performance, which is the honest caveat: your SQL still runs, it may simply run at a different speed. Module 5 exists because of that gap.

## Rule 9 - Logical data independence

**Demands:** applications must not be affected by information-preserving changes to the logical design, such as splitting or combining tables.

**At ICICI Bank:** adding `risk_rating` to `customer` must not break the teller screen.

**Reality:** partly honoured, and **you decide which half you get**. An application selecting named columns survives. An application using `SELECT *` receives a different result set the moment a column is added. This is the concrete reason the course bans `SELECT *` in application code.

## Rule 10 - Integrity independence

**Demands:** integrity constraints must be definable in the language and stored in the catalog, not in application programs.

**At ICICI Bank:** `balance >= 0` belongs in a `CHECK` constraint. `account.customer_id` must point at a real customer, enforced by a foreign key. Then the rule holds whether the write arrives from the mobile app, the ATM switch, a batch job or a DBA at a prompt.

**Reality:** honoured, and widely ignored by teams who put the rules in application code instead. Module 4 is largely an argument for this rule, and Module 12 fails any capstone that enforces integrity only in Python.

## Rule 11 - Distribution independence

**Demands:** applications must not be affected when data is distributed across several machines, or later redistributed.

**At ICICI Bank:** moving reporting queries to a read replica, or sharding transaction history by region, should change no SQL.

**Reality: the least honoured rule.** Replication, sharding and federation all leak into the application through connection strings, read-after-write staleness and distributed transaction limits. Module 9's replication section shows exactly where the illusion breaks.

## Rule 12 - The non-subversion rule

**Demands:** if the system provides a low-level, row-at-a-time interface, that interface must not be able to bypass the integrity rules expressed in the relational language.

**At ICICI Bank:** a bulk loader must not be allowed to insert a transaction whose `account_id` matches no account. If it can, every constraint in the database is a suggestion.

**Reality:** honoured in principle, deliberately subverted in practice. `COPY`, `BULK INSERT` and `bcp` all offer ways to defer or disable constraint checking for speed, and Module 11's migration uses exactly that, loading first and adding constraints afterwards. That is a controlled, temporary exception, with reconciliation afterwards to prove nothing invalid survived.

## What to take from this

| Rule          | Status in real products                        | Where the course returns to it |
|---------------|------------------------------------------------|--------------------------------|
| 1, 2, 4, 5, 7 | Solid                                          | Modules 2, 3, 9, 11            |
| 3             | Honoured, and the biggest source of quiet bugs | Modules 2, 3                   |
| 6             | Frequently broken                              | Module 5, views                |
| 8             | Honoured for correctness, not for speed        | Module 5                       |

| Rule | Status in real products                          | Where the course returns to it |
|------|--------------------------------------------------|--------------------------------|
| 9    | Yours to keep or lose, via <code>SELECT *</code> | Module 2 onwards               |
| 10   | Honoured, often ignored by teams                 | Modules 4 and 12               |
| 11   | Largely aspirational                             | Module 9                       |
| 12   | Honoured, deliberately suspended for loads       | Module 11                      |

**The two to remember under pressure** are rule 2 and rule 10. Rule 2 says that if you know the table, the key and the column, you can always reach the data. Rule 10 says that the rules protecting that data belong in the database, where every application meets them, rather than in one application that happens to be well behaved.

## 3. The database landscape at ICICI Bank

A real bank does not have one database. It has dozens, acquired at different times for different reasons.

| System                                | Database             | Why                                                         |
|---------------------------------------|----------------------|-------------------------------------------------------------|
| Core banking (accounts, transactions) | Oracle               | Came with the purchased core banking product, must run 24x7 |
| Digital lending platform              | PostgreSQL           | Built in-house, no licence cost, strong SQL support         |
| Internet banking session store        | MySQL                | Bundled with the web stack, simple read-heavy workload      |
| Regulatory reporting warehouse        | Microsoft SQL Server | Runs the bank's BI tooling and SSIS pipelines               |
| Balance and profile cache             | Redis                | Module 7                                                    |
| Transaction event backbone            | Apache Kafka         | Module 8                                                    |

### 3.1 Oracle Database

Oracle is the most common core banking database in Indian and global banking, largely for historical and support reasons.

#### Strengths that matter to a bank

- **Real Application Clusters (RAC)** - several servers share one database, so a node failure does not stop the bank.
- **Partitioning** - a transaction table with billions of rows is split by month, so queries and archival touch only what they need.
- **PL/SQL** - a complete procedural language inside the database. Much banking business logic lives here.
- **Data Guard and Flashback** - standby databases and the ability to query the past, both valuable during incidents.

#### Editions

| Edition                     | Notes                                                                               |
|-----------------------------|-------------------------------------------------------------------------------------|
| Enterprise Edition          | Full feature set, licensed per processor core, expensive                            |
| Standard Edition 2          | Cheaper, capped at a number of sockets, no partitioning                             |
| <b>Express Edition (XE)</b> | Free, capped at 12 GB of user data and 2 GB of RAM. This is what you install today. |

Because Oracle is licensed per core, banks are careful about what they let run on it. That constraint is a large part of why reporting is moved off Oracle, and why migration projects like Module 11 happen.

### 3.2 PostgreSQL

PostgreSQL is open source under a permissive licence, with no per-core cost, and it has become the default choice for new banking systems.

#### Strengths

- Standards-compliant SQL with excellent support for window functions and common table expressions, both covered in Module 6.
- **MVCC** concurrency - readers never block writers and writers never block readers. Module 6 explains how.
- **JSONB** columns with indexing, useful for KYC document payloads and API responses.
- A large extension ecosystem, including `pg_stat_statements` for finding slow queries in Module 5.

PostgreSQL is the **primary teaching database** for this programme. Almost everything you learn transfers to Oracle and SQL Server with small syntax adjustments, which are pointed out as you go.

## 3.3 MySQL

MySQL is the most widely deployed open-source database in web applications.

### Strengths

- Very fast for read-heavy, simple-lookup workloads such as session and profile data.
- Simple to install, replicate and operate.
- The **InnoDB** storage engine provides full ACID transactions, row-level locking and foreign keys.

**One warning that matters in banking:** the older **MyISAM** storage engine supports neither transactions nor foreign keys. A crash mid-write leaves a corrupt table with no rollback. MyISAM must never hold money. Always confirm the engine:

```
SELECT table_name, engine
FROM   information_schema.tables
WHERE  table_schema = 'icicibank';
-- every row must say InnoDB
```

## 3.4 Choosing for a workload

| Workload                                                   | Reasonable choice    | Reason                                 |
|------------------------------------------------------------|----------------------|----------------------------------------|
| Posting 2,000 transactions per second                      | Oracle or PostgreSQL | Proven OLTP throughput and durability  |
| Storing a customer's KYC JSON document                     | PostgreSQL           | Native JSONB with indexing             |
| Serving 50,000 concurrent web sessions                     | MySQL, or Redis      | Simple key lookups at high read volume |
| Quarterly NPA and regulatory reporting                     | SQL Server warehouse | Columnstore indexes, SSIS, BI tooling  |
| Sub-millisecond balance lookup                             | Redis                | Module 7                               |
| One transaction event consumed by fraud, alerts and ledger | Kafka                | Module 8                               |

## 4. OLTP and OLAP

### 4.1 OLTP - running the bank

**Online Transaction Processing** is the day-to-day operational work: a withdrawal, a transfer, a balance enquiry, a new account.

```
19:42:07 Card inserted, ATM at Andheri East
19:42:19 Customer requests 5,000
19:42:19 BEGIN
        SELECT balance FROM account WHERE account_id = 9002;
        UPDATE account SET balance = balance - 5000 WHERE account_id = 9002;
        INSERT INTO account_txn (...) VALUES (... , 'DEBIT', 5000, 'ATM', ...);
        COMMIT
19:42:20 Cash dispensed
```

Characteristics: a handful of rows touched, sub-second response required, very high transaction rate, a mix of reads and writes, and data that is always current.

### 4.2 OLAP - understanding the bank

**Online Analytical Processing** is analysis: trends, aggregates, comparisons, regulatory reporting.

```
-- Which branches grew deposits fastest this quarter?
SELECT  b.branch_name,
        SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE 0 END) AS inflow,
        SUM(CASE WHEN t.txn_type = 'DEBIT' THEN t.amount ELSE 0 END) AS outflow,
        COUNT(*) AS txn_count
FROM    account_txn t
        JOIN account a ON a.account_id = t.account_id
        JOIN branch b ON b.branch_id = a.branch_id
WHERE   t.txn_date >= DATE '2026-01-01'
GROUP BY b.branch_name
ORDER BY inflow DESC;
```

Characteristics: millions of rows scanned, read only, seconds to minutes of runtime, low query rate, and data that is often a day old.

### 4.3 The comparison

|                | OLTP                                 | OLAP                                |
|----------------|--------------------------------------|-------------------------------------|
| Purpose        | Run the business                     | Analyse the business                |
| Users          | Customers, tellers, ATMs, UPI apps   | Analysts, risk officers, regulators |
| Rows per query | 1 to 10                              | Millions                            |
| Response time  | Milliseconds                         | Seconds to minutes                  |
| Query rate     | Thousands per second                 | A few per hour                      |
| Operations     | INSERT, UPDATE, DELETE, point SELECT | Large SELECT with GROUP BY          |
| Schema design  | Normalized (Module 4)                | Star or snowflake, denormalized     |

|                 | OLTP                             | OLAP                       |
|-----------------|----------------------------------|----------------------------|
| Data currency   | Real time                        | Often loaded overnight     |
| Backup priority | Point-in-time recovery essential | Reload from source if lost |

## 4.4 Why they are separated

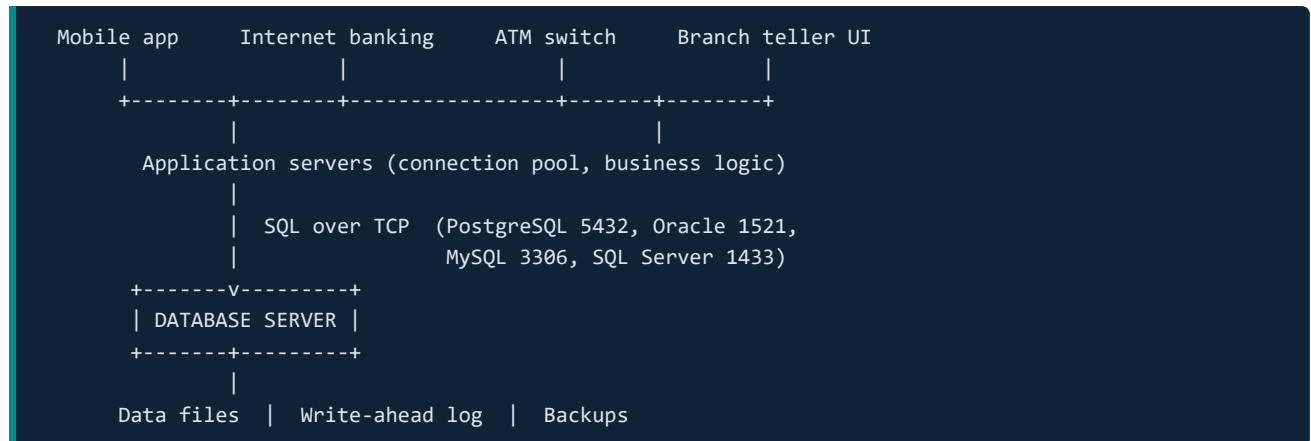
A report that scans forty million transaction rows takes memory, holds read locks and saturates disk. While it runs, ATM withdrawals queue behind it. Customers see "service temporarily unavailable" and the bank sees a spike in call-centre volume.

ICICI Bank's rule is therefore: **the core banking database serves customers and nothing else**. Reports run against a replica or against the SQL Server warehouse, loaded overnight by SSIS.

That single operational rule is the reason three later modules exist: replication in Module 9, the SQL Server warehouse in Module 10, and the ETL and migration work in Module 11.

## 5. Database architecture

### 5.1 Client-server



The database is a shared service used by many applications at once. It cannot assume any one of them is well behaved, which is why integrity rules and permissions live inside the engine rather than in application code.

### 5.2 The three-schema architecture

ANSI/SPARC describes a database at three levels. The value of this idea is **isolation of change**.

| Level             | What it describes                                      | Who cares             | ICICI Bank example                                                                |
|-------------------|--------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------|
| <b>External</b>   | What one group of users may see                        | Application developer | A teller's view shows name and balance; an auditor's view shows every transaction |
| <b>Conceptual</b> | The logical design: tables, columns, keys, constraints | Data engineer, DBA    | The nine-table core banking schema                                                |
| <b>Internal</b>   | Physical storage: files, blocks, indexes, compression  | DBA, storage admin    | <code>account_txn</code> partitioned by month on SSD                              |

### 5.3 Data independence

- **Physical data independence** - you can change how data is stored without changing the logical design or any application. Moving `account_txn` to faster storage, or adding an index, changes no SQL.
- **Logical data independence** - you can change the logical design without changing applications. Adding a `risk_rating` column to `customer` does not break existing screens, provided they select named columns rather than `SELECT *`.

**A practical consequence:** never write `SELECT *` in application code. It destroys logical data independence, because adding a column changes what the application receives.

### 5.4 Inside the engine

#### Processes

| Process               | Job                                                      |
|-----------------------|----------------------------------------------------------|
| Listener / postmaster | Accepts connections and creates a session                |
| Query executor        | Parses, plans and runs SQL for one session               |
| Background writer     | Flushes modified pages from memory to data files         |
| Checkpoint            | Periodically marks a point from which recovery can start |
| Log writer            | Writes the write-ahead log to disk, in order             |

## Memory

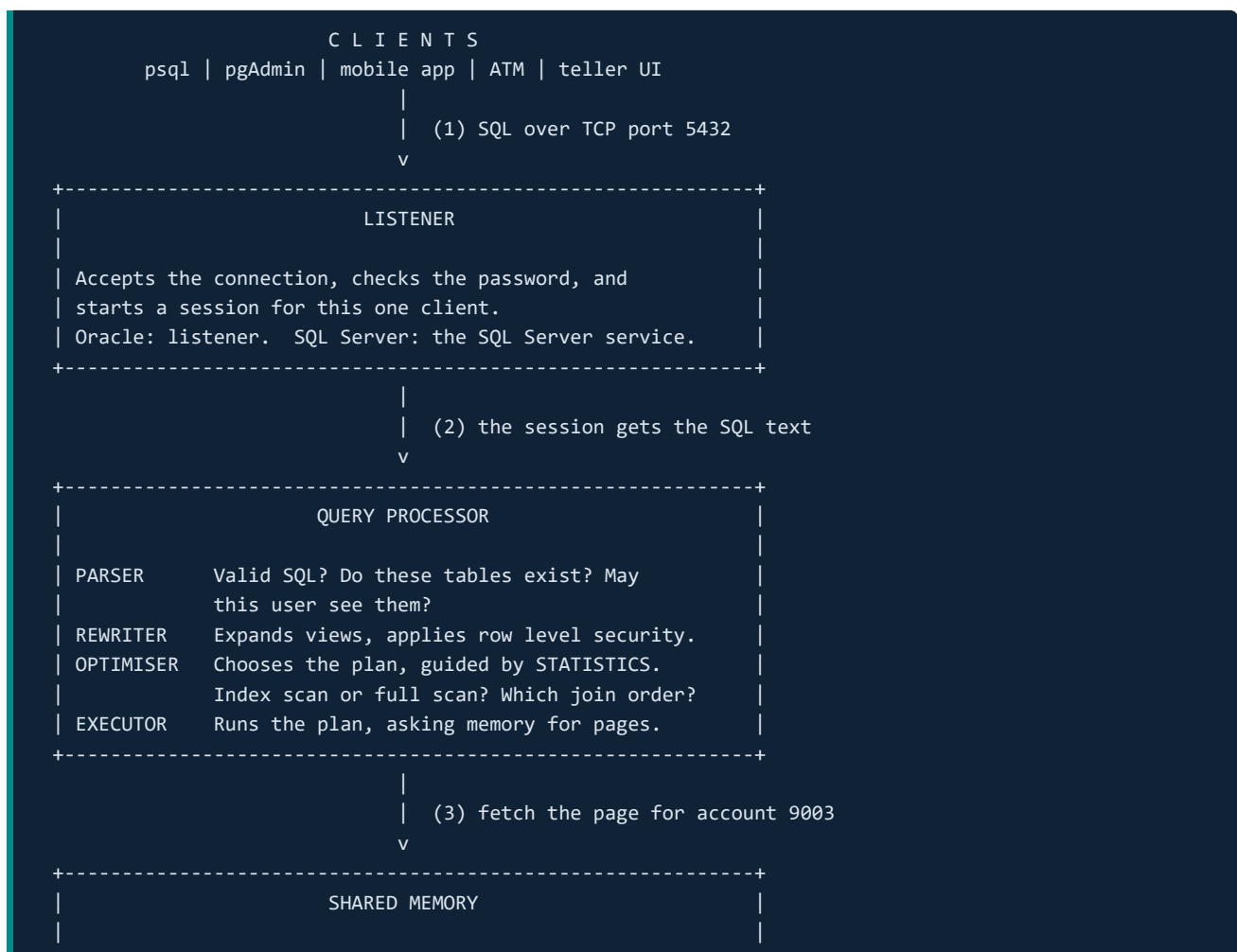
The engine keeps recently used data pages in a shared cache so that repeated reads never touch disk. Oracle calls this the **SGA**, PostgreSQL calls it **shared\_buffers**, SQL Server calls it the **buffer pool**. A well-sized cache is often the single largest performance factor on a database server.

## Storage

Rows are packed into fixed-size **pages** - 8 KB in PostgreSQL and SQL Server, a configurable block size in Oracle. A table is a collection of pages; an index is a B-tree of pages pointing into it. You will work directly with this structure in Module 5.

## 5.5 The engine in one picture

Everything named above, and how a balance enquiry travels through it.



```

| BUFFER CACHE  Recently used 8 KB pages.          |
|               HIT -> answered, no disk touched.   |
|               MISS -> the page is read from disk. |
|               shared_buffers | SGA | buffer pool   |
| WAL BUFFER    Change records awaiting their flush. |
| PLAN CACHE    Statements already parsed and planned. |
| LOCK TABLE  Who holds what, and who is waiting.  |
| WORK MEM      Per session space for sorts and hashes. |
+-----+
|
| (4) dirty pages and WAL go to disk
|
v
+-----+
|               BACKGROUND PROCESSES               |
|               |                                   |
| BACKGROUND WRITER Trickle dirty pages out to disk. |
| WAL WRITER       Flushes the WAL, always on COMMIT. |
| CHECKPOINTER    Marks a point recovery can start at. |
| AUTOVACUUM      Reclaims space from dead row versions. |
| ARCHIVER        Copies finished WAL files away.      |
| STATS COLLECTOR Feeds the optimiser its statistics.  |
+-----+
|
| (5) the only durable place
|
v
+-----+
|               DISK                                |
|               |                                   |
| DATA FILES     The tables and the indexes themselves. |
| WAL / REDO      Every change, written BEFORE the data page. |
| ARCHIVE         Copied WAL. The input to Module 9 recovery. |
| TEMP FILES      Sorts and hashes too large for work mem. |
| CONTROL FILE    Where the engine is in the log.        |
+-----+

```

## Reading a balance

Steps 1 to 3, then straight back out. If the page holding account 9003 is already in the buffer cache, **no disk is touched at all**. That is why the cache is usually the single largest performance factor on a database server, and it is the same idea you will put in front of the database with Redis in Module 7.

## Writing a transfer

A **COMMIT** does **not** write your data page to disk. It writes the **WAL record** and waits for that to be safely on disk. The data page is written later, by the background writer, at a time that suits the engine. This is the ordering rule in 5.6, and it is the whole reason a bank can promise durability without waiting for slow random writes.

## The same components, three vendors

| Component       | PostgreSQL                  | Oracle           | SQL Server      |
|-----------------|-----------------------------|------------------|-----------------|
| Buffer cache    | <code>shared_buffers</code> | SGA buffer cache | Buffer pool     |
| Log buffer      | WAL buffer                  | Redo log buffer  | Log buffer      |
| Transaction log | WAL                         | Redo logs        | Transaction log |

| Component              | PostgreSQL            | Oracle        | SQL Server       |
|------------------------|-----------------------|---------------|------------------|
| Log writer             | WAL writer            | LGWR          | Log writer       |
| Page flusher           | Background writer     | DBWn          | Lazy writer      |
| Checkpoint             | Checkpointer          | CKPT          | Checkpoint       |
| Space reclaim          | Autovacuum            | Undo and SMON | Ghost cleanup    |
| Per session sort space | <code>work_mem</code> | PGA           | Workspace memory |

**The names differ, the architecture does not.** Learn it once here and Module 10's SQL Server internals and Module 11's Oracle source are both already familiar.

## 5.6 The life of a query

1. **Parse** - is the SQL syntactically valid, do the tables and columns exist, is this user permitted?
2. **Rewrite** - expand views, apply row-level security policies.
3. **Optimise** - the planner estimates costs and chooses a plan: which index, which join order, which join algorithm. This is the step you learn to influence in Module 5.
4. **Execute** - the executor runs the plan, reading pages from the buffer cache or from disk.
5. **Fetch** - rows are returned to the client, usually in batches.

The same SQL text can produce different plans over time as the data grows and statistics change. This surprises people, and it is the root cause of many "it was fast yesterday" incidents.

## 5.7 Write-ahead logging

The single most important rule in database durability:

**The log record is written to disk before the data page is.**

When you update a balance, the engine first appends a record to the write-ahead log describing the change, and flushes that to disk. Only later does it write the modified data page. If the server crashes in between, restart replays the log and the change is recovered.

PostgreSQL calls this the **WAL**. Oracle calls it **redo logs**. SQL Server calls it the **transaction log**. The mechanism is the same, and it is what makes the D in ACID possible.

## 6. ACID

### 6.1 Why a bank cares

#### Incident ICICI-9902

14:22:31 Customer 1001 transfers 25,000 to customer 1007. Account 9001 is debited, balance now 153,450.  
14:22:32 the database server loses power. 14:22:58 the server restarts. 15:05 customer 1007 calls to say the money never arrived. 15:40 customer 1001 confirms it left their account. 25,000 rupees now exist nowhere.

ACID is the set of guarantees that makes this outcome impossible. A **transaction** is a unit of work that the database treats as indivisible.

### 6.2 Atomicity

A transaction is all or nothing. There is no such thing as a half-completed transfer.

```
BEGIN;  
  UPDATE account SET balance = balance - 25000 WHERE account_id = 9001;  
  UPDATE account SET balance = balance + 25000 WHERE account_id = 9007;  
COMMIT;    -- both changes become permanent together
```

If the second statement fails, or the connection drops, or the server dies, the engine undoes the first update during recovery. The money is never in flight.

You choose the outcome explicitly:

| Statement                 | Effect                                                  |
|---------------------------|---------------------------------------------------------|
| BEGIN / START TRANSACTION | Opens a transaction                                     |
| COMMIT                    | Makes every change in the transaction permanent         |
| ROLLBACK                  | Discards every change in the transaction                |
| SAVEPOINT name            | Marks a point you can roll back to partially (Module 6) |

### 6.3 Consistency

A transaction takes the database from one valid state to another valid state. "Valid" means every declared rule still holds: primary keys, foreign keys, check constraints, unique constraints, and any triggers.

```
ALTER TABLE account  
  ADD CONSTRAINT ck_balance_non_negative CHECK (balance >= 0);  
  
UPDATE account SET balance = balance - 500000 WHERE account_id = 9001;  
-- ERROR: new row for relation "account" violates check constraint  
--        "ck_balance_non_negative"
```

Consistency is not something the engine invents. **You get consistency by declaring rules that the engine then enforces.** A database with no constraints is perfectly happy to hold nonsense. This is the theme of Module 4.

## 6.4 Isolation

Concurrent transactions must not see each other's incomplete work. Consider a joint account with 10,000 rupees and two ATM withdrawals of 8,000 at the same instant.

| Without isolation                      | With isolation                    |
|----------------------------------------|-----------------------------------|
| -----                                  | -----                             |
| T1 reads balance 10,000                | T1 reads 10,000 and locks the row |
| T2 reads balance 10,000                | T2 waits for the lock             |
| T1 writes $10,000 - 8,000 = 2,000$     | T1 writes 2,000 and commits       |
| T2 writes $10,000 - 8,000 = 2,000$     | T2 now reads 2,000                |
| Both withdrawals succeed               | T2 needs 8,000, only 2,000 exists |
| Bank paid 16,000 from a 10,000 balance | T2 is declined. Correct.          |

The database offers a choice of **isolation levels**, trading strictness against concurrency. Module 6 covers them, along with locks, deadlocks and MVCC.

## 6.5 Durability

Once **COMMIT** returns successfully, the change survives anything short of losing the disks: a power cut, a crash, a kernel panic.

The engine achieves this by refusing to acknowledge the commit until the write-ahead log record is physically on disk. Data pages may still be sitting dirty in memory - that is fine, because the log can rebuild them.

On restart the engine performs **recovery**: it replays log records for committed transactions (redo) and reverses changes belonging to transactions that never committed (undo).

**A rule worth carrying into your career:** "the application printed success" means nothing. "COMMIT returned without error" is a promise the database is contractually bound to keep.

## 6.6 What each property protects

| Property    | If broken                                       | Consequence at a bank                             |
|-------------|-------------------------------------------------|---------------------------------------------------|
| Atomicity   | Money debited but never credited                | Customer complaint, regulator-reportable incident |
| Consistency | Negative balances, transactions with no account | Audit qualification, unreliable reporting         |
| Isolation   | Two withdrawals against the same balance        | Direct financial loss                             |
| Durability  | A committed transfer vanishes after a crash     | Total loss of trust, potential licence risk       |

## 7. Command reference for the lab

### PostgreSQL - psql

| Command                                    | Purpose                                 |
|--------------------------------------------|-----------------------------------------|
| <code>psql -U postgres -h localhost</code> | Connect                                 |
| <code>\l</code>                            | List databases                          |
| <code>\c icicibank</code>                  | Connect to a database                   |
| <code>\dt</code>                           | List tables                             |
| <code>\d account</code>                    | Describe the <code>account</code> table |
| <code>\dn</code>                           | List schemas                            |
| <code>\du</code>                           | List users and roles                    |
| <code>\timing on</code>                    | Show how long each statement takes      |
| <code>\q</code>                            | Quit                                    |

### Oracle - SQL\*Plus

| Command                                                    | Purpose                      |
|------------------------------------------------------------|------------------------------|
| <code>sqlplus system/password@localhost:1521/XEPDB1</code> | Connect                      |
| <code>SELECT name FROM v\$database;</code>                 | Current database             |
| <code>SELECT username FROM all_users;</code>               | List users (schemas)         |
| <code>SELECT table_name FROM user_tables;</code>           | List your tables             |
| <code>DESC account</code>                                  | Describe a table             |
| <code>SHOW PARAMETER sga_target</code>                     | Inspect memory configuration |

### MySQL

| Command                                  | Purpose           |
|------------------------------------------|-------------------|
| <code>mysql -u root -p</code>            | Connect           |
| <code>SHOW DATABASES;</code>             | List databases    |
| <code>USE icicibank;</code>              | Select a database |
| <code>SHOW TABLES;</code>                | List tables       |
| <code>DESCRIBE account;</code>           | Describe a table  |
| <code>SHOW ENGINE INNODB STATUS\G</code> | Engine internals  |

### Terms introduced in this module

| Term                                     | Meaning                                                     |
|------------------------------------------|-------------------------------------------------------------|
| DBMS                                     | Software that stores data and controls all access to it     |
| RDBMS                                    | A DBMS that stores data as related tables and enforces keys |
| Relation, tuple, attribute               | Table, row, column                                          |
| Primary key                              | Uniquely identifies a row                                   |
| Foreign key                              | Must match a primary key in another table                   |
| OLTP                                     | Short, frequent operational transactions                    |
| OLAP                                     | Large analytical queries over historical data               |
| Three-schema architecture                | External, conceptual and internal levels                    |
| Data independence                        | Changing one level without disturbing another               |
| Buffer cache / SGA / shared_buffers      | In-memory cache of data pages                               |
| Write-ahead log / redo / transaction log | Record of changes, written before data pages                |
| ACID                                     | Atomicity, Consistency, Isolation, Durability               |
| Transaction                              | An indivisible unit of database work                        |

## Where this module leads

| You learned                    | It becomes                                      |
|--------------------------------|-------------------------------------------------|
| Tables, rows, columns          | Module 2, where you create them                 |
| Primary and foreign keys       | Module 4, where you design with them            |
| Buffer cache and pages         | Module 5, where you tune around them            |
| Transactions and isolation     | Module 6, where you control them                |
| Why OLTP and OLAP are separate | Modules 9, 10 and 11                            |
| ACID guarantees                | Module 12, where your capstone must honour them |